

ONBOARDING GUIDE

ansuman/RepoLens

GitHub repository summaries and code Q&A

Repository	https://github.com/ansuman/RepoLens
Branch	main
Commit	3f9c2a17b64e0d5c8a1b2f3e4d5c6b7a8e9f0a1b
Primary language	C#
Files analyzed	142
Generated	25 September 2026, 14:50 UTC

Contents

1.	What this project does	2
2.	Tech stack	2
3.	Architecture	2
4.	Module map	2
5.	Key flows	3
6.	Entry points	3
7.	Data model and storage	3
8.	Configuration	4
9.	Build, run and test	4
10.	Where to start reading	4
11.	Glossary	4

Generated automatically from the repository's source. Verify important details against the code.

What this project does

RepoLens is a .NET 10 backend that lets people sign in with GitHub, see all their repositories, generate an onboarding PDF for any repository, and ask questions about its code.

Summaries are produced by an event-driven background job so the API always answers in milliseconds.

Who it is for: Frontend developers integrating the API, and backend developers maintaining the summary pipeline.

Tech stack

Technology	Category	Used for
ASP.NET Core 10	Framework	REST API, authentication, rate limiting, OpenAPI
MongoDB	Database	Users, repositories, jobs, code chunks, conversations and PDFs (GridFS)
Azure Service Bus	Messaging	Queue of summary commands and topic of completion events
Microsoft.Extensions.AI + OpenAI SDK	Library	Chat and embedding models (Azure OpenAI or OpenAI)
QuestPDF	Library	Renders the onboarding PDF

Architecture

The solution follows Clean Architecture. Domain holds entities such as SummaryJob with its state machine. Application holds use cases and the ports (interfaces) they need. Infrastructure implements those ports with MongoDB, GitHub, Service Bus, OpenAI and QuestPDF.

Two hosts share the code: RepoLens.Api answers HTTP requests and runs the outbox dispatcher; RepoLens.Worker consumes GenerateSummary commands and runs the six-step summary pipeline.

Module map

Folder	Responsibility	Key files
src/RepoLens.Domain	Entities and rules: User, Repo, SummaryJob, Conversation, CodeChunk.	Summaries/SummaryJob.cs Summaries/RepoSummary.cs
src/RepoLens.Application	Use cases, ports, prompts, the pipeline steps and the job processor.	Summaries/Processing/ SummaryJobProcessor.cs Questions/ QuestionAnsweringService.cs
src/ RepoLens.Infrastructure	MongoDB repositories, GitHub client, messaging, AI wiring, PDF renderer.	DependencyInjection.cs Messaging/ServiceBus/ ServiceBusConsumer.cs

Folder	Responsibility	Key files
src/RepoLens.Api	Controllers, GitHub OAuth + JWT, Problem Details, Swagger.	Program.cs Controllers/ SummariesController.cs
src/RepoLens.Worker	Service Bus consumer host for summary jobs.	Program.cs

Key flows

Summarize a repository

The Summarize button creates a job and a background worker turns the repository into a PDF.

1. POST /api/v1/repos/{id}/summaries reaches SummariesController.Start.
2. SummaryRequestService stores a queued SummaryJob (a unique index allows one active job per repository) and publishes GenerateSummaryCommand.
3. The API returns 202 Accepted with a Location header to poll.
4. The Worker's SummaryJobProcessor claims the job and runs the pipeline: resolve commit, download zipball, index code, generate summary, render PDF, store PDF.
5. The job is marked Completed and SummaryCompletedEvent is published on the summary-events topic.

Ask a question

Answers are grounded in code chunks retrieved by vector search.

1. POST /api/v1/repos/{id}/questions reaches QuestionsController.Ask.
2. QuestionAnsweringService embeds the question and searches code_chunks for the indexed commit.
3. QuestionPrompts builds a prompt with numbered <source> blocks; the model cites them as [n].
4. CitationMapper turns the markers into GitHub links with line anchors, and the exchange is saved to the conversation.

Entry points

File	Why it matters
src/RepoLens.Api/Program.cs	Composes the API: layers, authentication, rate limiting, Swagger, health checks.
src/RepoLens.Worker/Program.cs	Composes the Worker that consumes the summary-requests queue.

Data model and storage

MongoDB collections: users, repos, repo_indexes, summary_jobs, code_chunks (with 1536-dimension embeddings), conversations, auth_codes and refresh_tokens (both with TTL indexes), dataprotection_keys, and the summary_pdfs GridFS bucket.

Configuration

Setting	What it controls
Mongo:ConnectionString	MongoDB, Atlas or Cosmos DB for MongoDB vCore connection string.
GitHub:ClientId / ClientSecret	OAuth App credentials for GitHub sign-in.
Messaging:Provider	InMemory for local runs, ServiceBus in Azure.
Ai:Endpoint / ApiKey / ChatModel	Azure OpenAI or OpenAI settings.

Build, run and test

1. Start a local MongoDB

```
dotnet run --project tools/RepoLens.DevMongo
```

2. Run the API with Swagger UI

```
dotnet run --project src/RepoLens.Api
```

3. Run the tests

```
dotnet test
```

Where to start reading

1. src/RepoLens.Api/Program.cs
2. src/RepoLens.Domain/Summaries/SummaryJob.cs
3. src/RepoLens.Application/Summaries/Processing/SummaryJobProcessor.cs

Glossary

Term	Meaning
Outbox	The job document records whether its command reached the broker; a dispatcher re-publishes any that did not.
Dead-letter queue	Where Service Bus moves a message after 5 failed deliveries.